

머신러닝

판다스

Chapter 4



Prof. Yang



한국성서대학교
KOREAN BIBLE UNIVERSITY

목 차

CONTENTS

01 판다스란?

02 데이터 추출

03 그룹별 집계

04 데이터 결합

01 판다스란?



판다스의 개념

- 판다스(pandas) : 파이썬의 데이터 분석 라이브러리
 - 데이터 테이블(Data Table)을 다루는 도구
- 기본적으로 넘파이를 사용
 - 넘파이 : 파이썬에서 배열을 다루는 최적의 라이브러리
 - 판다스: 넘파이를 효율적으로 사용하기 위해 인덱싱, 연산, 전처리 등 다양한 함수 제공



판다스의 주요 객체

- 시리즈(Series) : 각 열 데이터를 다루는 객체 => 피쳐 벡터(Feature Vector)와 같은 개념
 - 리스트, 튜플, 넘파이 배열의 1차원 자료구조로, 데이터프레임의 한 행이나 열을 의미
 - list, dict, ndarray 등 다양한 데이터 타입이 시리즈 객체 형태로 변환되기도 함
- 데이터프레임(DataFrame) : 데이터 테이블 전체 객체

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	weight_0
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.9	4.98	1
1	0.02731	0.0	7.07	0	0.469	6.421	48.9	4.9671	2	242.0	17.8	396.9	9.14	1
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	1
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	1
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.9	5.33	1

시리즈(Series)

데이터프레임 중 하나의 열에
해당하는 데이터의 모음 객체

데이터프레임(DataFrame)

데이터 테이블 전체를
포함하는 객체

그림 4-1 데이터프레임 객체와 시리즈 객체



시리즈 객체

- 시리즈 객체를 생성하면 세 가지 요소(property) 생성

① 데이터(data) : 기존 다른 객체처럼 값을 저장하는 요소

② 인덱스(index)

- 시리즈 객체에서는 숫자, 문자열, 0 외의 값으로 시작하는 숫자(순서가 일정하지 않을 수 있음)
- 시리즈 객체에서는 인덱스 값의 중복을 허용

③ 데이터 타입(data type)

- 판다스는 넘파이의 래퍼(wrapper) 라이브러리
- 넘파이 배열을 기반으로 동작하며 넘파이 데이터 타입을 사용

인덱스(index) 값(values)

A	→	5
B	→	6
C	→	12
D	→	-5
E	→	6.7

- 시리즈(Series) 객체는 넘파이 배열(ndarray)의 하위 클래스
- 넘파이가 지원하는 어떠한 데이터 타입도 지원
- 인덱스와 반드시 정렬되어 있을 필요는 없음
- 인덱스 값은 중복을 허용



데이터프레임 객체

- 데이터 테이블 전체를 지칭하는 객체
- 넘파이 배열을 기반으로 동작
- 인덱싱 : 열과 행 각각 인덱스를 사용하여 하나의 데이터에 접근
- 열 단위로 추가 삭제 등 수정 가능
 - 'read_확장자' 함수로 데이터 바로 로딩
 - .csv나 .xlsx 등 스프레드시트형 확장자 파일에서 데이터 로딩

인덱스	열	foo	bar	baz	qux
		↓	↓	↓	↓
A	→	0	x	2.7	True
B	→	4	y	6	True
C	→	8	z	10	False
D	→	-12	w	NA	False
E	→	16	a	18	False

```
import pandas as pd# (1) pandas 모듈 호출
```

```
#(2)데이터 URL을 변수 data_url에 넣기
```

```
data_url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/housing/housing.data'
```

```
#(3) csv 데이터 로드
```

```
df_data = pd.read_csv(data_url, sep='\s+', header = None)
```

Ch2. 데이터의 이해 p.25

- '\s+' 공백이 하나 이상 있는 부분기준으로 데이터를 나누어 읽음
- 데이터에 열 이름이 없으면 header = None



데이터프레임 객체 생성

- 딕셔너리 타입 데이터에서 키(key)는 열 이름, 값(value)은 시퀀스형 데이터 타입을 넣어 각 열의 데이터로 만듦

```
from pandas import DataFrame

raw_data = {'first_name': ['Jason', 'Molly', 'Tina', 'Jake', 'Amy'],
            'last_name': ['Miller', 'Jacobson', 'Ali', 'Milner', 'Cooze'],
            'age': [42, 52, 36, 24, 73],
            'city': ['San Francisco', 'Baltimore', 'Miami', 'Douglas', 'Boston']}

df = pd.DataFrame(raw_data, columns = ['first_name', 'last_name', 'age', 'city'])
df
```

	first_name	last_name	age	city
0	Jason	Miller	42	San Francisco
1	Molly	Jacobson	52	Baltimore
2	Tina	Ali	36	Miami
3	Jake	Milner	24	Douglas
4	Amy	Cooze	73	Boston



데이터프레임의 열 다루기

- 데이터 생성시, 열 이름을 한정하면 해당 열만 추출

```
DataFrame(raw_data, columns = ["age", "city"])
```

	age	city
0	42	San Francisco
1	52	Baltimore
2	36	Miami
3	24	Douglas
4	73	Boston

- 데이터가 존재하지 않는 열을 추가하면 해당 열에는 NaN 값들 추가

```
DataFrame(raw_data,  
          columns = ["first_name", "last_name", "age", "city", "debt"])
```

	first_name	last_name	age	city	debt
0	Jason	Miller	42	San Francisco	NaN
1	Molly	Jacobson	52	Baltimore	NaN
2	Tina	Ali	36	Miami	NaN
3	Jake	Milner	24	Douglas	NaN
4	Amy	Cooze	73	Boston	NaN

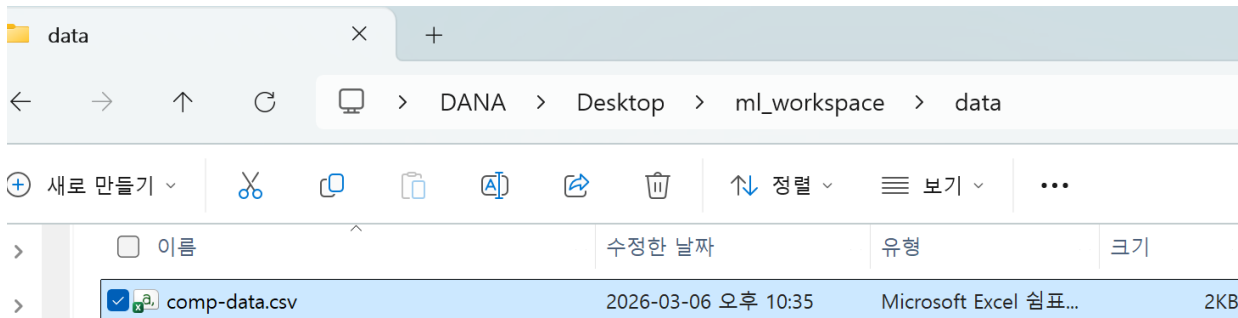
02

데이터 추출



데이터 로딩

- “~/ml_workspace/data/comp-data.csv”데이터로 실습 진행
 - 경로 유의하기



```
file = './data/comp-data.csv'
```

```
df_data = pd.read_csv(file) # csv 데이터 로드
df = pd.DataFrame(df_data)
df
```

	account	name	street	city	state	postal-code	Jan	Feb	Mar
0	211829	Kerluke, Koepp and Hilpert	34456 Sean Highway	New Jaycob	Texas	28752	10000	62000	35000
1	320563	Walter-Trantow	1311 Alvis Tunnel	Port Khadijah	NorthCarolina	38365	95000	45000	35000
2	648336	Bashirian, Kunde and Price	62184 Schamberger Underpass Apt. 231	New Lilianland	Iowa	76517	91000	120000	35000
3	109996	D'Amore, Gleichner and Bode	155 Fadel Crescent Apt. 144	Hyattburgh	Maine	46021	45000	120000	10000
4	121213	Bauch-Goldner	7274 Marissa Common	Shanahanchester	California	49681	162000	120000	35000
5	132071	Willms, Schulz and Hutter	68102 Gail - Garcia	Smithburgh	Alaska	62705	150000	120000	35000



데이터 프레임 확인

- head와 tail 함수 : 처음 n개 행이나 마지막 n개 행 호출

```
df.head()#default는 5개
```

	account	name	street	city	state	postal-code	Jan	Feb	Mar
0	211829	Kerluke, Koepp and Hilpert	34456 Sean Highway	New Jaycob	Texas	28752	10000	62000	35000
1	320563	Walter-Trantow	1311 Alvis Tunnel	Port Khadijah	NorthCarolina	38365	95000	45000	35000
2	648336	Bashirian, Kunde and Price	62184 Schamberger Underpass Apt. 231	New Lilianland	Iowa	76517	91000	120000	35000
3	109996	D'Amore, Gleichner and Bode	155 Fadel Crescent Apt. 144	Hyattburgh	Maine	46021	45000	120000	10000
4	121213	Bauch-Goldner	7274 Marissa Common	Shanahanchester	California	49681	162000	120000	35000

```
df.head(3).T
```

	0	1	2
account	211829	320563	648336
name	Kerluke, Koepp and Hilpert	Walter-Trantow	Bashirian, Kunde and Price
street	34456 Sean Highway	1311 Alvis Tunnel	62184 Schamberger Underpass Apt. 231
city	New Jaycob	Port Khadijah	New Lilianland
state	Texas	NorthCarolina	Iowa
postal-code	28752	38365	76517
Jan	10000	95000	91000
Feb	62000	45000	120000
Mar	35000	35000	35000

```
df.tail(4)
```

	account	name	street	city	state	postal-code	Jan	Feb	Mar
10	214098	Goodwin, Homenick and Jerde	649 Cierra Forks Apt. 078	Rosaberg	Tennessee	47743	45000	120000	55000
11	231907	Hahn-Moore	18115 Olivine Throughway	Norbertomouth	NorthDakota	31415	150000	10000	162000
12	242368	Frami, Anderson and Donnelly	182 Bertie Road	East Davian	Iowa	72686	162000	120000	35000
13	268755	Walsh-Haley	2624 Beatty Parkways	Goodwinmouth	Rhodelsland	31919	55000	120000	35000
14	273274	McDermott PLC	8917 Bergstrom Meadow	Kathryneborough	Delaware	27933	150000	120000	70000



열 or 행 호출

- 열 이름을 리스트 형태로 넣어 호출
 - 가장 일반적인 호출 방법
 - 문자형 열 이름을 하나만 넣으면 값이 시리즈 객체로 반환됨
 - 열 이름을 여러 개 넣으면 데이터프레임 객체로 반환됨

```
df[["account", "street", "state"]].head(3)
```

	account	street	state
0	211829	34456 Sean Highway	Texas
1	320563	1311 Alvis Tunnel	NorthCarolina
2	648336	62184 Schamberger Underpass Apt. 231	Iowa

- 행 인덱스 번호로 호출
 - 기존의 리스트나 넘파이 배열(ndarray) 인덱싱과 동일

```
df[:3]
```



	account	name	street	city	state	postal-code	Jan	Feb	Mar
0	211829	Kerluke, Koepp and Hilpert	34456 Sean Highway	New Jaycob	Texas	28752	10000	62000	35000
1	320563	Walter-Trantow	1311 Alvis Tunnel	Port Khadijah	NorthCarolina	38365	95000	45000	35000
2	648336	Bashirian, Kunde and Price	62184 Schamberger Underpass Apt. 231	New Lilianland	Iowa	76517	91000	120000	35000



행과 열을 모두 사용한 데이터 추출

- 데이터의 일정 부분을 사각형 형태로 잘라냄

1) 위의 두 가지 방법(열 이름과 행 번호) 함께 사용

```
df[["name", "city", "state"]][7:13]
```

2) iloc 함수 : 인덱스 번호로만 데이터 호출

- 열 이름 사용 할 수 없음

- .iloc[행, 열]

```
df.iloc[7:13, [1,3,4]]
```

- 위 코드 결과 둘 다 아래와 같음

	name	city	state
7	Kovacek-Johnston	Deronville	Rhodelsland
8	Champlin-Morar	Lake Juliannton	Pennsylvania
9	Gerhold-Maggio	North Ras	Idaho
10	Goodwin, Homenick and Jerde	Rosaberg	Tenessee
11	Hahn-Moore	Norbertomouth	NorthDakota
12	Frami, Anderson and Donnelly	East Davian	Iowa



데이터 삭제

- drop 함수 : 특정 열이나 행을 삭제한 데이터 프레임 객체를 반환
 - 행 삭제

Ch02 데이터의 이해 p28.pop함수
:pop함수는 열 제거 후 해당 열 반환

```
df = df.drop(1) #행 1번 삭제
df.head()
```

	account	name	street	city	state	postal-code	Jan	Feb	Mar
0	211829	Kerluke, Koepp and Hilpert	34456 Sean Highway	New Jaycob	Texas	28752	10000	62000	35000
2	648336	Bashirian, Kunde and Price	62184 Schamberger Underpass Apt. 231	New Lilianland	Iowa	76517	91000	120000	35000
3	109996	D'Amore, Gleichner and Bode	155 Fadel Crescent Apt. 144	Hyattburgh	Maine	46021	45000	120000	10000
4	121213	Bauch-Goldner	7274 Marissa Common	Shanahanchester	California	49681	162000	120000	35000
5	132971	Williamson, Schumm and Hettinger	89403 Casimer Spring	Jeremieburgh	Arkansas	62785	150000	120000	35000

인덱스 번호는 바뀌지 않음

```
import numpy as np
df.drop(np.arange(4,10), inplace=True)#inplace 원본 데이터프레임에서 바로 삭제
df
```

	account	name	street	city	state	postal-code	Jan	Feb	Mar
0	211829	Kerluke, Koepp and Hilpert	34456 Sean Highway	New Jaycob	Texas	28752	10000	62000	35000
2	648336	Bashirian, Kunde and Price	62184 Schamberger Underpass Apt. 231	New Lilianland	Iowa	76517	91000	120000	35000
3	109996	D'Amore, Gleichner and Bode	155 Fadel Crescent Apt. 144	Hyattburgh	Maine	46021	45000	120000	10000
10	214098	Goodwin, Homenick and Jerde	649 Cierra Forks Apt. 078	Rosaberg	Tennessee	47743	45000	120000	55000
11	231907	Hahn-Moore	18115 Olivine Throughway	Norbertomouth	NorthDakota	31415	150000	10000	162000
12	242368	Frami, Anderson and Donnelly	182 Bertie Road	East Davian	Iowa	72686	162000	120000	35000
13	268755	Walsh-Haley	2624 Beatty Parkways	Goodwinmouth	RhodeIsland	31919	55000	120000	35000
14	273274	McDermott PLC	8917 Bergstrom Meadow	Kathryneborough	Delaware	27933	150000	120000	70000



데이터 삭제

- drop 함수 : 특정 열이나 행을 삭제한 객체를 반환
 - 열 삭제

```
df.drop("account",axis=1).head(3) # account 열 제거
```

	name	street	city	state	postal-code	Jan	Feb	Mar
0	Kerluke, Koepp and Hilpert	34456 Sean Highway	New Jaycob	Texas	28752	10000	62000	35000
2	Bashirian, Kunde and Price	62184 Schamberger Underpass Apt. 231	New Lilianland	Iowa	76517	91000	120000	35000
3	D'Amore, Gleichner and Bode	155 Fadel Crescent Apt. 144	Hyattburgh	Maine	46021	45000	120000	10000

```
df.drop(["account", "name"], axis=1,inplace = True) # account, name 열 바로 삭제
df.head(3)
```

	street	city	state	postal-code	Jan	Feb	Mar
0	34456 Sean Highway	New Jaycob	Texas	28752	10000	62000	35000
2	62184 Schamberger Underpass Apt. 231	New Lilianland	Iowa	76517	91000	120000	35000
3	155 Fadel Crescent Apt. 144	Hyattburgh	Maine	46021	45000	120000	10000



조건 필터링

- 조건식이 True인 인덱스의 행들만 선택

```
df[df['Jan']>= 150000]
```

	account	name	street	city	state	postal-code	Jan	Feb	Mar
4	121213	Bauch-Goldner	7274 Marissa Common	Shanahanchester	California	49681	162000	120000	35000
5	132971	Williamson, Schumm and Hettinger	89403 Casimer Spring	Jeremieburgh	Arkansas	62785	150000	120000	35000
11	231907	Hahn-Moore	18115 Olivine Throughway	Norbertomouth	NorthDakota	31415	150000	10000	162000
12	242368	Frami, Anderson and Donnelly	182 Bertie Road	East Davian	Iowa	72686	162000	120000	35000
14	273274	McDermott PLC	8917 Bergstrom Meadow	Kathryneborough	Delaware	27933	150000	120000	70000

```
df[(df['Jan']>= 10000) & (df['state'] == 'Iowa')]
```

	account	name	street	city	state	postal-code	Jan	Feb	Mar
2	648336	Bashirian, Kunde and Price	62184 Schamberger Underpass Apt. 231	New Lilianland	Iowa	76517	91000	120000	35000
12	242368	Frami, Anderson and Donnelly	182 Bertie Road	East Davian	Iowa	72686	162000	120000	35000



정렬

■ sort_values()로 데이터를 원하는 순서로 재배열

- by

- 기준값

- 스칼라값, 벡터값 둘다 가능

- ascending

- 오름차순: True가 디폴트

- 내림차순: False

```
df_sort = df.sort_values(by="state")#오름차순
df_sort.head()
```

	account	name	street	city	state	postal-code	Jan	Feb	Mar
5	132971	Williamson, Schumm and Hettinger	89403 Casimer Spring	Jeremieburgh	Arkansas	62785	150000	120000	35000
4	121213	Bauch-Goldner	7274 Marissa Common	Shanahanchester	California	49681	162000	120000	35000
14	273274	McDermott PLC	8917 Bergstrom Meadow	Kathryneborough	Delaware	27933	150000	120000	70000
9	212303	Gerhold-Maggio	366 Maggio Grove Apt. 998	North Ras	Idaho	46308	70000	120000	35000
2	648336	Bashirian, Kunde and Price	62184 Schamberger Underpass Apt. 231	New Lilianland	Iowa	76517	91000	120000	35000

```
df.sort_values(by="city", ascending=False).head()#내림차순
```

	account	name	street	city	state	postal-code	Jan	Feb	Mar
4	121213	Bauch-Goldner	7274 Marissa Common	Shanahanchester	California	49681	162000	120000	35000
10	214098	Goodwin, Homenick and Jerde	649 Cierra Forks Apt. 078	Rosaberg	Tennessee	47743	45000	120000	55000
1	320563	Walter-Trantow	1311 Alvis Tunnel	Port Khadijah	NorthCarolina	38365	95000	45000	35000
9	212303	Gerhold-Maggio	366 Maggio Grove Apt. 998	North Ras	Idaho	46308	70000	120000	35000
11	231907	Hahn-Moore	18115 Olivine Throughway	Norbertomouth	NorthDakota	31415	150000	10000	162000

```
df.sort_values(by=["city", "name"]).head(3)
```

	account	name	street	city	state	postal-code	Jan	Feb	Mar
7	205217	Kovacek-Johnston	91971 Cronin Vista Suite 601	Deronville	Rhodelsland	53461	145000	95000	35000
12	242368	Frami, Anderson and Donnelly	182 Bertie Road	East Davian	Iowa	72686	162000	120000	35000
13	268755	Walsh-Haley	2624 Beatty Parkways	Goodwinmouth	Rhodelsland	31919	55000	120000	35000

03

그룹별 집계

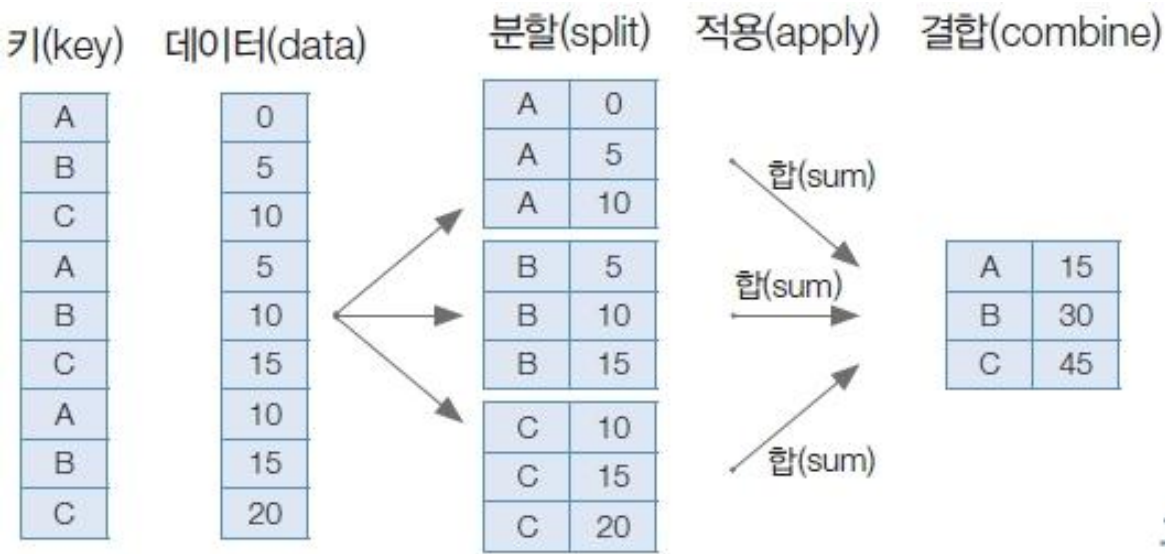


그룹별 집계 개념

■ 그룹별 집계(groupby) : 데이터로부터 동일한 객체를 가진 데이터만 따로 뽑아 기술통계 데이터를 추출

• 예) A반 수학 점수의 원본 데이터(raw data)를 가지고 있을 때 해당 데이터에서

- 같은 성별을 가진 학생들의 평균 점수 계산
- 50점 이상을 받은 학생의 수를 계산



• groupby 명령어는 분할→적용→결합 과정을 거침

- 분할(split) : 같은 종류의 데이터끼리 나누는 기능
- 적용(apply) : 데이터 블록마다 sum, count, mean 등 연산 적용
- 결합(combine) : 연산 함수가 적용된 각 블록들을 합침



그룹별 집계的基本형

```
import pandas as pd # pandas 모듈 호출
import numpy as np # numpy 모듈 호출

ipl_data = {'Team': ['Riders', 'Riders', 'Devils', 'Devils', 'Kings', 'kings', 'Kings', 'Kings', 'Riders', 'Royals', 'Royals', 'Riders'],
            'Rank': [1, 2, 2, 3, 3, 4, 1, 1, 2, 4, 1, 2],
            'Year': [2014, 2015, 2014, 2015, 2014, 2015, 2016, 2017, 2016, 2014, 2015, 2017],
            'Points': [876, 789, 863, 673, 741, 812, 756, 788, 694, 701, 804, 690]}

df = pd.DataFrame(ipl_data)
df
```

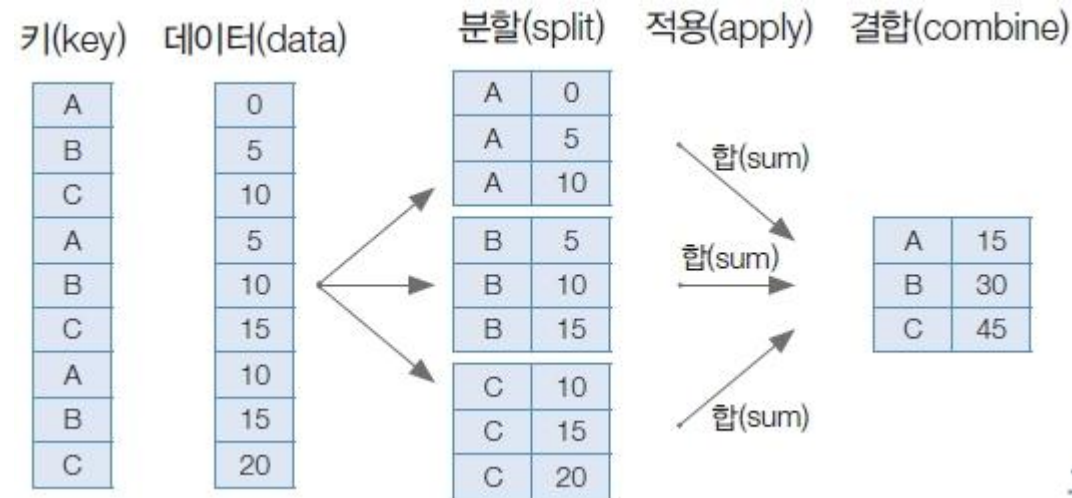
	Team	Rank	Year	Points
0	Riders	1	2014	876
1	Riders	2	2015	789
2	Devils	2	2014	863
3	Devils	3	2015	673
4	Kings	3	2014	741
5	kings	4	2015	812
6	Kings	1	2016	756
7	Kings	1	2017	788
8	Riders	2	2016	694
9	Royals	4	2014	701
10	Royals	1	2015	804
11	Riders	2	2017	690

```
df.groupby("Team")["Points"].sum()
```

```
Team
Devils    1536
Kings     2285
Riders    3049
Royals    1505
kings      812
Name: Points, dtype: int64
```



그룹별 집계的基本형



1. 키로 분할

2. 추출 할 데이터 분할

3. 적용 및 결합

`df.groupby("Team")["Points"].sum()`

묶음의 기준이 되는 열

적용받는 열

적용받는 연산



멀티 인덱스 그룹별 집계

- 한 개 이상의 열을 기준으로 그룹별 집계를 실행
 - 리스트를 사용하여 여러 개의 열 이름을 기준으로 넣으면 여러 열이 키 값이 되어 시리즈 객체로 결과 출력
 - 계층적 인덱스(hierarchical index) 형태 : level로 나누어짐

```
multi_groupby = df.groupby(["Team", "Year"])["Points"].sum()  
multi_groupby
```

	level 1	level 0
Team	Year	
Devils	2014	863
	2015	673
Kings	2014	741
	2016	756
	2017	788
Riders	2014	876
	2015	789
	2016	694
Royals	2017	690
	2014	701
	2015	804
kings	2015	812

Name: Points, dtype: int64

```
multi_groupby["Riders"]
```

Year

2014	876
2015	789
2016	694
2017	690

Name: Points, dtype: int64

```
multi_groupby["Kings":"Royals"]
```

Team	Year	
Kings	2014	741
	2016	756
	2017	788
Riders	2014	876
	2015	789
	2016	694
Royals	2017	690
	2014	701
	2015	804

Name: Points, dtype: int64



멀티 인덱스 그룹별 집계

- 각 레벨에 별도의 연산함수를 적용할 수 있음

```
multi_groupby.groupby(level=0).sum()
```

```
Team
Devils      1536
Kings       2285
Riders      3049
Royals      1505
kings        812
Name: Points, dtype: int64
```

```
multi_groupby.groupby(level=1).sum()
```

```
Year
2014      3181
2015      3078
2016      1450
2017      1478
Name: Points, dtype: int64
```

04 데이터 결합



병합

- 병합(merge) : 두 개의 데이터를 특정 기준을 가지고 하나로 통합하는 작업

	A	B
1	ID	Price
2	1001	2
3	1002	3
4	1003	2
5	1004	4
6	1005	1

+

	A	B	C
1	ID	Store	Location
2	1001	A	N
3	1003	A	N
4	1004	C	W
5	1002	B	E
6	1005	C	W

=

	A	B	C	D
1	ID	Price	Store	Location
2	1001	2	A	N
3	1002	3	B	E
4	1003	2	A	N
5	1004	4	C	W
6	1005	1	C	W

- 데이터 준비

```
import pandas as pd
raw_data = {
    'subject_id': ['1', '2', '3', '4', '5', '7', '8', '9', '10', '11'],
    'test_score': [51, 15, 15, 61, 16, 14, 15, 1, 61, 16]}

df_left = pd.DataFrame(raw_data, columns = ['subject_id', 'test_score'])
df_left
```

```
raw_data = {
    'subject_id': ['4', '5', '6', '7', '8'],
    'first_name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],
    'last_name': ['Bonder', 'Black', 'Balwner', 'Brice', 'Btisan']}

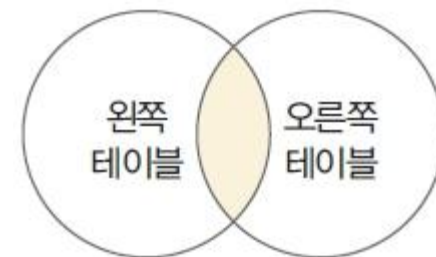
df_right = pd.DataFrame(raw_data, columns = ['subject_id', 'first_name', 'last_name'])
df_right
```



내부 조인

- 내부 조인(inner join) : 키 값을 기준으로 두 테이블에 모두 존재하는 키 값의 행끼리 병합
- 집합으로 보면 양쪽의 교집합 데이터를 통합
- subject_id를 기준으로 내부 조인을 수행
 - 키 값 subject_id 열의 값이 두 테이블 모두 존재해야 병합됨

내부 조인(inner join)



```
pd.merge(left=df_left, right=df_right, how="inner", on='subject_id')
```

	subject_id	test_score	first_name	last_name
0	4	61	Billy	Bonder
1	5	16	Brian	Black
2	7	14	Bryce	Brice
3	8	15	Betty	Btisan

- left, right 매개변수에 각 위치에 해당하는 데이터프레임 객체를 입력
- how에 조인 방법 "inner"를 문자열 타입으로 입력
- on에 병합의 기준이 되는 열 이름을 입력

left_on과 right_on 매개변수

- 왼쪽 테이블과 오른쪽 테이블의 키 값이 다른 경우
left_on과 right_on 매개변수를 사용하여 각 테이블 키 값을 입력

```
pd.merge(left=df_left, right=df_right,  
         left_on='subject_id', right_on='subject_id')
```



완전 조인

- 완전 조인(full join) : 두 테이블의 합집합을 의미
 - 양쪽에 같은 키 값이 있는 데이터는 합치고 나머지는 NaN

```
pd.merge(df_left, df_right, on='subject_id', how='outer')
```

	subject_id	test_score	first_name	last_name
0	1	51.0	NaN	NaN
1	10	61.0	NaN	NaN
2	11	16.0	NaN	NaN
3	2	15.0	NaN	NaN
4	3	15.0	NaN	NaN
5	4	61.0	Billy	Bonder
6	5	16.0	Brian	Black
7	6	NaN	Bran	Balwner
8	7	14.0	Bryce	Brice
9	8	15.0	Betty	Btisan
10	9	1.0	NaN	NaN



연결

- 연결(concatenate) : 두 테이블을 그대로 붙임
 - concat 함수 : 두 개의 서로 다른 테이블을 하나로 합침

```
import os
path = "./data"
filenames = [os.path.join(path, filename)
              for filename in os.listdir(path) if "sales" in filename]
print(filenames) #파일 이름
```

```
[ './data\\sales-feb-2014.csv', './data\\sales-jan-2014.csv', './data\\sales-mar-2014.csv' ]
```

```
df_list = [pd.read_csv(f) for f in filenames] #filenames 기반으로 read_csv

for df in df_list:
    print(type(df), len(df))
```

```
<class 'pandas.core.frame.DataFrame'> 108
<class 'pandas.core.frame.DataFrame'> 134
<class 'pandas.core.frame.DataFrame'> 142
```

```
df = pd.concat(df_list, axis=0) #axis=0으로 vertical연결
df = df.reset_index(drop=True) #중복된 인덱스를 제거 및 인덱싱 다시하기
print(len(df))
```

Hierarchical연결 시 concat함수를 사용하는 것보다
아래 표현이 오류가 적음

```
import numpy as np
df['new'] = np.empty(len(df)) #hierarchical연결
#df[['new1', 'new2']] = new_df
```



SUMMARY

- 판다스?파이썬에서 데이터 테이블을 다루는 데이터 분석 라이브러리
 - 넘파이 배열을 기반으로 동작
 - Series : 1차원 데이터 구조
 - DataFrame : 행과 열로 구성된 데이터 테이블
- 데이터 추출
 - 데이터 로딩 : read_csv()
 - 데이터 확인 : head(), tail()
- 데이터 선택
 - 열 선택 : df[['col1','col2']]
 - 행 선택 : df[0:3]
 - 위치 기반 추출 : iloc[행, 열]
 - 조건 필터링 : df[df['col'] > value]
 - 정렬 : sort_values()
- 데이터 삭제 : drop()
- 데이터 분석
 - 그룹별 집계 : groupby()
- 데이터 결합
 - 병합 : merge()
 - 연결 : concat()



Q&A

궁금한 사항은 아래 방법으로 문의 바랍니다.



이메일

dana1112@bible.ac.kr



LMS 쪽지

THANK YOU FOR YOUR ATTENTION!

한국성서대학교 | 인공지능융합학부 | 머신러닝